CSE312: Database Management Systems Lab

Lab 04: PostgreSQL Data Query Language (DQL)

1. Objective

The objectives of this experiment are:

- To understand the purpose of Data Query Language (DQL).
- To retrieve selected columns, expressions, and distinct values with `SELECT`.
- To filter individual rows with a detailed understanding of the `WHERE` clause.
- To sort query results and return a controlled number of rows.
- To summarize data using aggregate functions.
- To form groups with `GROUP BY` and filter groups with `HAVING`.
- To combine query results using set operators.

2. Introduction to DQL

DQL: Data Query Language is the part of SQL used to read, filter, calculate, summarize, and present data without directly changing the stored rows.

The principal DQL command is `SELECT`. A simple `SELECT` reads one table, while an advanced `SELECT` may create groups, calculate aggregate values, combine result sets, or use the result of another query.

2.1 General SELECT Syntax

```
SELECT [DISTINCT] select_list
FROM source
[WHERE row_condition]
[GROUP BY grouping_columns]
[HAVING group_condition]
[ORDER BY sort_expression [ASC | DESC]]
[LIMIT row_count]
[OFFSET rows_to_skip];
```

Square brackets in the syntax mean that the enclosed part is optional. They should not be typed in an actual query.

Complete clause breakdown:

- `SELECT`: starts the query and specifies the values to display.
- `DISTINCT`: optionally removes duplicate output rows.
- `select_list`: contains columns, expressions, functions, or subqueries to return.
- `FROM`: identifies the table, view, or subquery that supplies rows.
- `WHERE`: filters individual source rows before grouping.
- `GROUP BY`: collects rows that have equal grouping values.
- `HAVING`: filters complete groups after aggregate values are calculated.
- `ORDER BY`: sorts the final result.
- `ASC`: requests ascending order; it is the default.
- `DESC`: requests descending order.
- `LIMIT`: restricts the maximum number of rows returned.
- `OFFSET`: skips a specified number of rows before returning results.
- The semicolon terminates the SQL statement.

2.2 Query Clauses at a Glance

Table 1: Important DQL clauses and keywords

Part	Function	Example
`SELECT`	Chooses the output columns or expressions.	`SELECT first_name, admission_year`
`DISTINCT`	Removes duplicate rows from the selected result.	`SELECT DISTINCT dept_id`
`AS`	Assigns a readable alias to a column or source.	`COUNT(*) AS total`
`FROM`	Identifies the source of the rows.	`FROM student`
`WHERE`	Filters individual rows.	`WHERE admission_year >= 2022`
`GROUP BY`	Forms one group for each distinct grouping value.	`GROUP BY dept_id`

Part	Function	Example
`HAVING`	Filters groups, usually using an aggregate.	`HAVING COUNT(*) >= 2`
`ORDER BY`	Sorts the result.	`ORDER BY admission_year DESC`
`LIMIT`	Sets the maximum number of returned rows.	`LIMIT 5`
`OFFSET`	Skips rows in the sorted result.	`OFFSET 5`
`UNION`	Combines results and removes duplicates.	`query_1 UNION query_2`
`UNION ALL`	Combines results and preserves duplicates.	`query_1 UNION ALL query_2`
`INTERSECT`	Returns rows present in both results.	`query_1 INTERSECT query_2`
`EXCEPT`	Returns rows from the first result that are absent from the second.	`query_1 EXCEPT query_2`

2.3 Written Order and Logical Processing Order

SQL is written in a human-readable order, but its clauses are conceptually processed in a different order.

Step	Clause	Conceptual operation
1	`FROM`	Build the source row set.
2	`WHERE`	Keep source rows whose row condition is true.
3	`GROUP BY`	Divide the remaining rows into groups.
4	`HAVING`	Keep groups whose group condition is true.
5	`SELECT`	Calculate and produce the requested output values.
6	`DISTINCT`	Remove duplicate output rows when requested.
7	`ORDER BY`	Sort the output rows.
8	`OFFSET`	Skip rows from the beginning of the sorted result.
9	`LIMIT`	Return no more than the requested number of rows.

Table 2: Conceptual processing order of a SELECT query

2.4 General Query Safety and Readability Rules

- List only the columns that are required instead of using `*` in final reports.
- Qualify a column with a table alias when more than one source contains that column name.
- Use parentheses to make mixed `AND` and `OR` conditions unambiguous.
- Use `IS NULL` or `IS NOT NULL` to test null values.
- Use `ORDER BY` with `LIMIT` when the identity of the selected rows matters.
- Give every subquery in the `FROM` clause an alias.
- Test an inner query separately before placing it inside a larger query.

3. Preparing the Database

This experiment uses the `university_lab` database and the four related tables introduced in Labs 02 and 03.

```
psql -U postgres
\c university_lab
```

If the tables are not available, create them with the following statements.

```
CREATE TABLE IF NOT EXISTS department (
    dept_id SERIAL PRIMARY KEY,
    dept_name VARCHAR(100) NOT NULL UNIQUE,
    location VARCHAR(100)
);

CREATE TABLE IF NOT EXISTS student (
    student_id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    date_of_birth DATE,
    admission_year INTEGER CHECK (admission_year >= 2000),
    is_active BOOLEAN DEFAULT TRUE,
    dept_id INTEGER REFERENCES department(dept_id)
);

CREATE TABLE IF NOT EXISTS course (
    course_id SERIAL PRIMARY KEY,
    course_code VARCHAR(20) NOT NULL UNIQUE,
    course_title VARCHAR(150) NOT NULL,
    credit NUMERIC(3, 1) CHECK (credit > 0),
    dept_id INTEGER NOT NULL REFERENCES department(dept_id)
);

CREATE TABLE IF NOT EXISTS enrollment (
    student_id INTEGER REFERENCES student(student_id),
    course_id INTEGER REFERENCES course(course_id),
    enrollment_date DATE DEFAULT CURRENT_DATE,
    grade VARCHAR(2),
    PRIMARY KEY (student_id, course_id),
    CONSTRAINT grade_check
        CHECK (grade IN ('A+', 'A', 'B+', 'B', 'C+', 'C', 'D', 'F'))
);
```

3.1 Loading Repeatable Sample Data

The following data gives the queries in this manual a predictable starting point. It deliberately includes repeated values, a student without a department, inactive students, and unassigned grades.

```
TRUNCATE TABLE enrollment, student, course, department
RESTART IDENTITY CASCADE;

INSERT INTO department (dept_name, location)
VALUES
    ('Computer Science and Engineering', 'Academic Building 1'),
    ('Electrical and Electronic Engineering', 'Academic Building 2'),
    ('Business Administration', 'Academic Building 3'),
    ('Mathematics', 'Academic Building 4');

INSERT INTO student
    (first_name, last_name, email, date_of_birth,
     admission_year, is_active, dept_id)
VALUES
    ('Amina', 'Rahman', 'amina.rahman@example.com',
     '2003-02-12', 2022, TRUE, 1),
    ('Nafis', 'Ahmed', 'nafis.ahmed@example.com',
     '2002-09-25', 2021, TRUE, 1),
    ('Sadia', 'Islam', 'sadia.islam@example.com',
     '2003-07-08', 2022, TRUE, 2),
    ('Tanvir', 'Hasan', 'tanvir.hasan@example.com',
     '2004-01-19', 2023, TRUE, 1),
    ('Maliha', 'Chowdhury', 'maliha.c@example.com',
     '2002-11-03', 2021, TRUE, 3),
    ('Rafi', 'Karim', 'rafi.karim@example.com',
     '2005-04-15', 2024, TRUE, 1),
    ('Nusrat', 'Jahan', 'nusrat.jahan@example.com',
     '2003-12-21', 2023, FALSE, 2),
    ('Imran', 'Hossain', 'imran.h@example.com',
     '2003-05-10', 2022, TRUE, 3),
    ('Farhan', 'Ali', 'farhan.ali@example.com',
     '2001-08-30', 2020, FALSE, 1),
    ('Tania', 'Sultana', 'tania.s@example.com',
     '2005-06-18', 2024, TRUE, NULL);
```

```
INSERT INTO course (course_code, course_title, credit, dept_id)
VALUES
    ('CSE312', 'Database Management Systems Lab', 1.5, 1),
    ('CSE311', 'Database Management Systems', 3.0, 1),
    ('CSE220', 'Data Structures', 3.0, 1),
    ('EEE201', 'Electrical Circuits', 3.0, 2),
    ('EEE202', 'Electronic Devices', 3.0, 2),
    ('BUS101', 'Introduction to Business', 3.0, 3),
    ('BUS201', 'Principles of Marketing', 3.0, 3),
    ('MAT101', 'Discrete Mathematics', 3.0, 4);

INSERT INTO enrollment
    (student_id, course_id, enrollment_date, grade)
VALUES
    (1, 1, '2026-01-15', 'A'),
    (1, 2, '2026-01-15', 'B+'),
    (2, 1, '2026-01-16', 'A+'),
    (2, 3, '2026-01-16', 'A'),
    (3, 4, '2026-01-17', 'B'),
    (3, 5, '2026-01-17', 'B+'),
    (4, 1, '2026-01-18', NULL),
    (4, 2, '2026-01-18', 'A'),
    (5, 6, '2026-01-19', 'A'),
    (5, 7, '2026-01-19', 'B'),
    (6, 1, '2026-01-20', 'B+'),
    (6, 3, '2026-01-20', 'A+'),
    (7, 4, '2026-01-21', 'C+'),
    (7, 5, '2026-01-21', NULL),
    (8, 6, '2026-01-22', 'B+'),
    (8, 7, '2026-01-22', 'A'),
    (9, 2, '2026-01-23', 'C'),
    (9, 8, '2026-01-23', 'B'),
    (10, 8, '2026-01-24', 'A');
```

4. Basic Data Retrieval with SELECT

4.1 Selecting All Columns

SELECT: The `SELECT` command retrieves rows and calculates the values that will appear in a query result.

General syntax:

```
SELECT *
FROM table_name;
```

```
SELECT *
FROM student;
```

Clause breakdown:

- `SELECT`: begins the retrieval operation.
- `*`: requests every column from the source.
- `FROM`: introduces the data source.
- `student`: is the table that supplies the rows.
- The semicolon ends the statement.

Using `*` is convenient for exploration. An explicit column list is usually clearer in reports and application code because it prevents unexpected output changes when the table structure changes.

4.2 Selecting Particular Columns and Assigning Aliases

```
SELECT
    student_id AS id,
    first_name AS given_name,
    last_name AS family_name
FROM student;
```

Clause breakdown:

- The comma-separated expressions after `SELECT` form the output column list.
- `student_id`, `first_name`, and `last_name` are source columns.
- `AS`: assigns an output alias without renaming the stored table column.
- `id`, `given_name`, and `family_name` are headings in the result.
- `FROM student`: reads rows from the `student` table.

4.3 Selecting Expressions

```
SELECT
    student_id,
    first_name || ' ' || last_name AS full_name,
    2026 - admission_year AS years_since_admission
FROM student;
```

Expression breakdown:

- `||`: concatenates text values in PostgreSQL.
- `' '`: supplies one space between the first and last names.
- `AS full_name`: labels the calculated text expression.
- `2026 - admission_year`: calculates a numeric value for each row.
- Expressions affect only the query output; they do not update stored data.

4.4 Removing Duplicate Results with DISTINCT

General syntax:

```
SELECT DISTINCT column_list
FROM table_name;
```

```
SELECT DISTINCT admission_year
FROM student
ORDER BY admission_year;
```

Clause breakdown:

- `DISTINCT`: removes duplicate combinations from the complete selected column list.
- `admission_year`: is the value whose distinct occurrences are displayed.
- `ORDER BY admission_year`: sorts the unique years in ascending order.
- If several columns follow `DISTINCT`, PostgreSQL compares the complete row of selected values, not each column independently.

5. Filtering Rows with the WHERE Clause

WHERE: The `WHERE` clause evaluates a true-or-false condition for every source row and keeps only rows for which the condition is true.

5.1 Basic Comparison Conditions

General syntax:

```
SELECT column_list
FROM table_name
WHERE condition;
```

```
SELECT student_id, first_name, last_name, admission_year
FROM student
WHERE admission_year >= 2022;
```

Clause breakdown:

- `SELECT`: chooses the columns to display.
- `FROM student`: identifies all student rows as the initial source.
- `WHERE`: starts the row-filtering condition.
- `admission_year >= 2022`: is true for students admitted in 2022 or later.
- Rows for which the condition is false or unknown are excluded.

Table 3: Common operators used in WHERE conditions

Operator	Meaning	Example
`=`	Equal to	`dept_id = 1`
`<> or !=`	Not equal to	`admission_year <> 2024`
`>`	Greater than	`credit > 1.5`
`>=`	Greater than or equal to	`admission_year >= 2022`
`<`	Less than	`admission_year < 2022`
`<=`	Less than or equal to	`credit <= 3.0`
`AND`	Both connected conditions must be true	`dept_id = 1 AND is_active`
`OR`	At least one connected condition must be true	`dept_id = 1 OR dept_id = 2`
`NOT`	Reverses a condition	`NOT is_active`
`BETWEEN`	Tests an inclusive range	`admission_year BETWEEN 2021 AND 2023`
`IN`	Tests membership in a value list or query result	`dept_id IN (1, 2)`
`LIKE`	Matches a case-sensitive text pattern	`first_name LIKE 'A%'`

Operator	Meaning	Example
`ILIKE`	Matches a case-insensitive text pattern	`email ILIKE '%@example.com'`
`IS NULL`	Tests for a null value	`grade IS NULL`
`IS NOT NULL`	Tests for a non-null value	`dept_id IS NOT NULL`

5.2 Combining Conditions with AND, OR, and NOT

```
SELECT student_id, first_name, admission_year, dept_id
FROM student
WHERE dept_id = 1
  AND (admission_year >= 2022 OR is_active = FALSE);
```

Condition breakdown:

- `dept_id = 1`: requires membership in department 1.
- The parenthesized expression accepts either a recent admission year or an inactive status.
- `AND`: requires the department condition and the parenthesized condition.
- `OR`: requires at least one condition inside the parentheses.
- `AND` has higher precedence than `OR`. Parentheses make the intended order explicit and reduce mistakes.

The following query demonstrates `NOT`.

```
SELECT student_id, first_name, is_active
FROM student
WHERE NOT is_active;
```

Because `is_active` is a Boolean column, `NOT is_active` selects rows whose value is `FALSE`.

5.3 Filtering a Range with BETWEEN

```
SELECT student_id, first_name, admission_year
FROM student
WHERE admission_year BETWEEN 2021 AND 2023;
```

Clause breakdown:

- `BETWEEN 2021 AND 2023`: includes both boundary values.
- The condition is equivalent to `admission_year >= 2021 AND admission_year <= 2023`.
- `NOT BETWEEN` may be used to select values outside the inclusive range.

5.4 Testing Membership with IN

```
SELECT course_code, course_title, dept_id
FROM course
WHERE dept_id IN (1, 3);
```

Clause breakdown:

- `IN`: compares one expression with every value in a list.
- `(1, 3)`: is the list of accepted department identifiers.
- The condition is equivalent to `dept_id = 1 OR dept_id = 3`.
- `NOT IN`: requests values that are not members of the list, but null values require special care.

5.5 Matching Text with LIKE and ILIKE

```
SELECT student_id, first_name, email
FROM student
WHERE first_name ILIKE 'a%';
```

Pattern breakdown:

- `ILIKE`: performs case-insensitive pattern matching in PostgreSQL.
- `'a%'`: matches text that begins with the letter `a`.
- The percent symbol `%` matches zero or more characters.
- The underscore `_` pattern symbol matches exactly one character.
- `LIKE` performs case-sensitive matching under common PostgreSQL collations.

5.6 Testing NULL Values

```
SELECT student_id, first_name, dept_id
FROM student
WHERE dept_id IS NULL;
```

Clause breakdown:

- `NULL`: represents missing, unknown, or inapplicable information.
- `IS NULL`: tests whether the value is null.
- `IS NOT NULL`: tests whether a known value exists.
- `dept_id = NULL` is not a valid replacement because ordinary comparison with null produces an unknown result.

SQL expresses what condition the rows must satisfy. PostgreSQL decides how to locate and process those rows efficiently.

6. Sorting and Limiting Query Results

6.1 Sorting with ORDER BY

General syntax:

```
SELECT column_list
FROM table_name
ORDER BY sort_expression [ASC | DESC], ...;
```

```
SELECT student_id, first_name, last_name, admission_year
FROM student
ORDER BY admission_year DESC, first_name ASC;
```

Clause breakdown:

- `ORDER BY`: sorts the final rows.
- `admission_year DESC`: places later admission years before earlier years.
- `first_name ASC`: breaks ties by first name in ascending order.
- `ASC`: is optional because ascending order is the default.
- Without `ORDER BY`, PostgreSQL does not guarantee a particular row order.

6.2 Returning a Fixed Number of Rows with LIMIT

LIMIT: The `LIMIT` clause sets the maximum number of rows that a query may return.

General syntax:

```
SELECT column_list
FROM table_name
ORDER BY sort_expression
LIMIT row_count;
```

```
SELECT student_id, first_name, admission_year
FROM student
ORDER BY admission_year DESC, student_id
LIMIT 3;
```

Clause breakdown:

- `ORDER BY admission_year DESC`: places the newest admission years first.
- `student_id`: provides a stable tie-breaker for students with the same year.
- `LIMIT 3`: returns no more than the first three rows of the sorted result.
- `LIMIT` does not mean "top" by itself; the meaning of "first" is defined by `ORDER BY`.

6.3 Skipping Rows with OFFSET

```
SELECT student_id, first_name, admission_year
FROM student
ORDER BY student_id
LIMIT 3 OFFSET 3;
```

Clause breakdown:

- `ORDER BY student_id`: establishes a repeatable order.
- `OFFSET 3`: skips the first three sorted rows.
- `LIMIT 3`: returns at most the next three rows.
- This pattern can implement simple page-based output.
- Large offsets can become inefficient because PostgreSQL still has to identify and skip the earlier rows.

7. Summarizing Data with Aggregate Functions

Aggregate Function: An aggregate function receives values from several rows and returns one summary value for the complete set or for each group.

7.1 Common Aggregate Functions

Table 4: Common aggregate functions

Function	Purpose	Example
`COUNT(*)`	Counts rows, including rows containing null values.	`COUNT(*) AS student_count`
`COUNT(column)`	Counts non-null values in one column.	`COUNT(grade) AS graded_count`
`COUNT(DISTINCT column)`	Counts distinct non-null values.	`COUNT(DISTINCT dept_id)`
`SUM(column)`	Adds numeric values.	`SUM(credit)`
`AVG(column)`	Calculates the arithmetic mean of non-null numeric values.	`AVG(credit)`
`MIN(column)`	Returns the minimum non-null value.	`MIN(admission_year)`
`MAX(column)`	Returns the maximum non-null value.	`MAX(admission_year)`

7.2 Aggregating the Complete Table

```
SELECT
    COUNT(*) AS total_students,
    COUNT(dept_id) AS students_with_department,
    MIN(admission_year) AS earliest_year,
    MAX(admission_year) AS latest_year
FROM student;
```

Clause breakdown:

- No `GROUP BY` appears, so the complete filtered table is treated as one group.
- `COUNT(*)`: counts every student row.
- `COUNT(dept_id)`: ignores rows whose department identifier is null.

- `MIN` and `MAX`: find the smallest and largest admission years.
- Each `AS` alias gives a descriptive heading to a calculated result.

7.3 Filtering Before Aggregation

```
SELECT COUNT(*) AS active_cse_students
FROM student
WHERE dept_id = 1 AND is_active = TRUE;
```

The `WHERE` clause first keeps active department 1 students. `COUNT(*)` then counts only those remaining rows.

8. Grouping Rows with GROUP BY and HAVING

8.1 Creating Groups with GROUP BY

GROUP BY: The `GROUP BY` clause collects rows with equal grouping values so that aggregate functions can calculate one result per group.

General syntax:

```
SELECT grouping_column, aggregate_function(...)
FROM table_name
[WHERE row_condition]
GROUP BY grouping_column
[ORDER BY sort_expression];
```

```
SELECT
    dept_id,
    COUNT(*) AS student_count,
    MIN(admission_year) AS earliest_admission
FROM student
GROUP BY dept_id
ORDER BY dept_id;
```

Clause breakdown:

- `FROM student`: supplies the source rows.
- `GROUP BY dept_id`: creates one group for each distinct department identifier, including a separate null group.

- `dept_id`: may appear in `SELECT` because it is a grouping column.
- `COUNT(*)`: counts rows inside each department group.
- `MIN(admission_year)`: calculates one earliest year for each group.
- `ORDER BY dept_id`: sorts the group result.

In a grouped query, every selected expression should normally be either:

- included in the `GROUP BY` clause, or
- calculated by an aggregate function.

8.2 Grouping by More Than One Column

```
SELECT dept_id, admission_year, COUNT(*) AS student_count
FROM student
GROUP BY dept_id, admission_year
ORDER BY dept_id, admission_year;
```

Clause breakdown:

- `GROUP BY dept_id, admission_year`: forms a group for each distinct combination.
- Students in the same department but different admission years belong to different groups.
- `COUNT(*)`: reports the size of each combination group.
- Both non-aggregate selected columns are listed in `GROUP BY`.

8.3 Filtering Groups with HAVING

HAVING: The `HAVING` clause evaluates a condition for each completed group and keeps only groups for which the condition is true.

General syntax:

```
SELECT grouping_column, aggregate_function(...)
FROM table_name
[WHERE row_condition]
GROUP BY grouping_column
HAVING group_condition
[ORDER BY sort_expression];
```

```
SELECT course_id, COUNT(*) AS enrollment_count
FROM enrollment
GROUP BY course_id
HAVING COUNT(*) >= 2
ORDER BY enrollment_count DESC, course_id;
```

Clause breakdown:

- `GROUP BY course_id`: creates one group for each course.
- `COUNT(*)`: calculates the number of enrollments in each course group.
- `HAVING COUNT(*) >= 2`: keeps only course groups containing at least two rows.
- `ORDER BY enrollment_count DESC`: sorts by the aggregate alias from largest to smallest.
- `course_id`: breaks ties in ascending order.

8.4 WHERE Compared with HAVING

```
SELECT dept_id, COUNT(*) AS active_student_count
FROM student
WHERE is_active = TRUE
GROUP BY dept_id
HAVING COUNT(*) >= 2
ORDER BY dept_id;
```

Clause breakdown:

- `WHERE is_active = TRUE`: removes inactive rows before groups are formed.
- `GROUP BY dept_id`: groups only the active rows.
- `HAVING COUNT(*) >= 2`: removes groups containing fewer than two active students.
- `WHERE` answers "which rows may enter a group?"
- `HAVING` answers "which completed groups may enter the result?"

Feature	WHERE	HAVING
Works on	Individual source rows	Completed groups
Logical timing	Before `GROUP BY`	After `GROUP BY`
Aggregate use	Normally not allowed	Common and expected
Typical condition	`is_active = TRUE`	`COUNT(*) >= 2`
Performance role	Reduces rows before grouping	Reduces groups after aggregation

Table 5: Comparison of WHERE and HAVING

9. Conditional and Null-Handling Expressions

9.1 Producing Categories with CASE

General syntax:

```
CASE
    WHEN condition_1 THEN result_1
    WHEN condition_2 THEN result_2
    ELSE default_result
END
```

Example usage:

```
SELECT
    student_id,
    first_name,
    admission_year,
    CASE
        WHEN admission_year >= 2023 THEN 'Recent'
        WHEN admission_year >= 2021 THEN 'Continuing'
        ELSE 'Senior'
    END AS student_category
FROM student
ORDER BY student_id;
```

Expression breakdown:

- `CASE`: begins a conditional expression.

- Each `WHEN`: states a condition tested from top to bottom.
- `THEN`: supplies the value for the first true condition.
- `ELSE`: supplies a value when no `WHEN` condition is true.
- `END`: closes the expression.

9.2 Replacing NULL for Display with COALESCE

```
SELECT
    student_id,
    first_name,
    COALESCE(email, 'No email recorded') AS contact_email
FROM student
ORDER BY student_id;
```

Expression breakdown:

- `COALESCE(value_1, value_2, ...)`: returns the first non-null argument.
- `email`: is used when the student record contains an email address.
- `'No email recorded'`: is returned when the email value is null.
- `COALESCE` changes the displayed result, not the stored value.

10. Combining Query Results with Set Operators

Set operators combine complete query results vertically. The participating queries must return the same number of columns, and corresponding columns must have compatible data types.

10.1 UNION and UNION ALL

```
SELECT first_name, last_name
FROM student
WHERE dept_id = 1

UNION

SELECT first_name, last_name
FROM student
```

```
WHERE admission_year = 2024
ORDER BY first_name, last_name;
```

Clause breakdown:

- The first `SELECT`: returns department 1 student names.
- The second `SELECT`: returns students admitted in 2024.
- `UNION`: combines both results and removes duplicate rows.
- `UNION ALL`: may replace `UNION` when duplicates should be preserved.
- The final `ORDER BY`: sorts the complete combined result.
- Output column names come from the first query.

10.2 INTERSECT

```
SELECT student_id
FROM student
WHERE dept_id = 1

INTERSECT

SELECT student_id
FROM enrollment
WHERE course_id = 1;
```

Clause breakdown:

- The first query returns identifiers of department 1 students.
- The second query returns identifiers of students enrolled in course 1.
- `INTERSECT`: keeps identifiers that occur in both query results.

10.3 EXCEPT

```
SELECT student_id
FROM student

EXCEPT

SELECT student_id
FROM enrollment;
```

Clause breakdown:

- The first query returns every student identifier.
- The second query returns identifiers that appear in enrollment records.
- `EXCEPT`: keeps students present in the first result but absent from the second.
- Reversing the query order changes the meaning of `EXCEPT`.

11. Reading a Complete DQL Query

The following query combines row filtering, grouping, group filtering, sorting, and limiting.

```
SELECT
    dept_id,
    admission_year,
    COUNT(*) AS active_students
FROM student
WHERE is_active = TRUE
  AND dept_id IS NOT NULL
GROUP BY dept_id, admission_year
HAVING COUNT(*) >= 2
ORDER BY active_students DESC, dept_id, admission_year
LIMIT 3;
```

Clause-by-clause interpretation:

1. `FROM student`: supplies the source rows.
2. `WHERE is_active = TRUE`: keeps only active students.
3. `dept_id IS NOT NULL`: removes students without an assigned department.
4. `GROUP BY dept_id, admission_year`: creates one group for each department and admission-year combination.
5. `COUNT(*) AS active_students`: counts the rows inside each group.
6. `HAVING COUNT(*) >= 2`: keeps only groups with at least two active students.
7. `ORDER BY`: ranks larger groups first, then applies stable tie-breakers.
8. `LIMIT 3`: returns at most the first three ranked groups.

12. Observations

- `SELECT` retrieved stored columns and calculated expressions without changing table data.
- `WHERE` filtered individual rows before grouping.
- `ORDER BY` established a defined result order.
- `LIMIT` and `OFFSET` selected a portion of a sorted result.
- Aggregate functions summarized complete tables and groups.
- `GROUP BY` created one result row for each distinct grouping value.
- `HAVING` filtered groups after aggregate values were available.
- Set operators combined compatible query results.
- Subqueries supplied values, sets, calculated columns, and table-like sources to outer queries.

13. Lab Task

Task A: Basic Retrieval and WHERE Practice

Using the `university_lab` database, write and execute queries to:

1. Display student identifiers, full names, and admission years.
2. Display all distinct admission years in descending order.
3. Find active students admitted from 2021 through 2023.
4. Find students whose first names begin with `M`, `N`, or `R`.
5. Find students who do not have a department.
6. Find courses belonging to departments 1, 2, or 4.
7. Add a short clause breakdown below each query.

Task B: Sorting, LIMIT, and OFFSET

1. Display the five most recently admitted students. Use a stable tie-breaker.
2. Display the second page of students when each page contains three rows.
3. Display the three courses with the highest credit values, sorted by course code when credits are equal.
4. Explain why each query requires `ORDER BY` before `LIMIT`.

Task C: Aggregate, GROUP BY, and HAVING

1. Count all students and count only students with assigned departments.
2. Show the number of students in each department.
3. Show the number of students for each department and admission year combination.
4. Show only departments containing at least two active students.
5. Show courses with at least three enrollments.
6. Show each course's total enrollment count and number of assigned grades.
7. Explain which conditions belong in `WHERE` and which belong in `HAVING`.

Task D: Online Shop DQL Problem

Assume an online shop database contains the tables `customer`, `product`, `orders`, and `order_item`. Write PostgreSQL queries to:

- display products whose price is between two chosen values;
- display the five products with the lowest stock;
- calculate the number and total value of items in each order;
- display only orders whose calculated value exceeds a chosen amount;
- display customers who have never placed an order;

14. Conclusion

This experiment introduced PostgreSQL Data Query Language through progressively structured `SELECT` statements. Students selected and calculated output values, filtered rows, sorted and limited results, summarized data, formed and filtered groups, and combined result sets.